CSC3002F – OS assignment 1 BHLJAY002

Understanding the simulation:

Before starting this assignment, I glossed over at the code provided, so that I could fully conceptualize what each object is and what parts of the computer each object refers to:

Barman (Sarah) – this is the CPU. It prepares the drinks (CPU is getting used) and waits for more orders from the patrons.

Patron – this is an entire process. Once all the patrons' orders are fulfilled, that is **one** process completed.

DrinkOrder – Each drink that a patron order is a job within a process. The drinks are analogous to a job.

Measuring metrics:

In this section of my report, I'm going to discuss what each of the metrics are in terms of the simulation, and how I measured these values.

- **Waiting time:** the time spent waiting for **all** the drinks to be served.

```
System.out.println("Order placed by " + drinksOrder[i].toString()); //output in standard format  - do not change this

long orderTime = System.currentTimeMillis(); //time for when drink is ordered
```

```
long waitedTime = System.currentTimeMillis(); //time for when the drink is given to the patron
if (i==0){
    responseTime = waitedTime - orderTime; //if its the first drink ordered, get the response time
    firstDrinkTime = orderTime;             //store time for when first drink is ordered for turnaround time
}
long waitTime = waitedTime - orderTime;
totalWaitingTime += waitTime;   //add all the waiting time for each drink ordered to get the total waiting time
```

I added timing signals right after a drink is ordered (**orderTime**) and right after a drink is received (**waitedTime**). I then cumulatively added these values for all the drinks ordered. This is how I obtained my values for the waiting time.

- **Response time:** the time spent waiting for the **1st drink** to be received.

```
long waitedTime = System.currentTimeMillis(); //time for when the drink is given to the patron
if (i==0){
    responseTime = waitedTime - orderTime; //if its the first drink ordered, get the response time
    firstDrinkTime = orderTime;             //store time for when first drink is ordered for turnaround time
}
```

To obtain this value, I added an if statement (to check ONLY for the first drink) and subtracted the **orderTime** from the **waitedTime**. I also retain the time the first drink is ordered(**firstDrinkTime**) so that I can get the turnaround time (see below).

- **Turnaround time:** the time it takes from when the first drink is ordered to when the last drink has been drunk.

```
        sleep(drinksOrder[i].getImbibingTime()); //drinking drink = "IO"X
    }
    long endTime = System.currentTimeMillis();
    turnaroundTime = endTime - firstDrinkTime; //get the turnaround time when the last drink is finished (ie the patrons work is done)

    System.out.println("Patron "+ this.ID + " completed ");
```

I obtained this value by waiting for the patron to finish all their drinks. When they are finished (after the loop), I obtain the time when the patron finished their last drink (**endTime**). I then subtract this time from the **firstDrinkTime** to get the turnaround time.

- **CPU Utilisation:** the ratio of the time that Sarah is busy preparing drinks to the total amount of time the simulation runs.

```
Sarah= new Barman(startSignal,sched,q,s);
Sarah.start();
//this is where simulation starts
startSimulationTime = System.currentTimeMillis();
```

```
//this is where simulation ends
endSimulationTime = System.currentTimeMillis();
System.out.println(x:"------Bar closed------");
totalSimulationTime = endSimulationTime - startSimulationTime;
```

To get the total amount of time the CPU is used (the total time the simulation runs), I obtain the times for when the simulation starts (**startSimulationTime**) and when the simulation ends (**endSimulationTime**). I subtract the start time from the end time to get the **totalSimulationTime**.

```
//CPU (barman) is getting used while preparing the drink - this is the time spent busy
long startBusyTime = System.currentTimeMillis();
sleep(currentOrder.getExecutionTime()); //processing order (="CPU burst")
long endBusyTime = System.currentTimeMillis();
//cumulatively add to totalBusyTime
totalBusyTime += (endBusyTime - startBusyTime);
```

For FCFS and SJF I add the total time the CPU (barman) is spent preparing drinks (as that is when the CPU is being used). This is how I obtain the value for time the CPU spends when its busy for FCFS and SFJ.

```
if(burst<=q) { //within the quantum
    long startBusyTime = System.currentTimeMillis();
    sleep(burst); //processing complete order ="CPU burst"
    long endBusyTime = System.currentTimeMillis();
    totalBusyTime += (endBusyTime - startBusyTime);
```

```
else {
    long startBusyTime = System.currentTimeMillis();
    sleep(q);
    long endBusyTime = System.currentTimeMillis();
    totalBusyTime += (endBusyTime - startBusyTime);
```

For RR I add the times spent when the CPU is used in bursts (each time quantum) and when the CPU is interrupted to process another job. I cumulatively add these values to **totalBusyTime.** This is how I obtain the value for time the CPU spends when its busy for RR.

```
double busyTime = (double)Sarah.getBusyTime();
double totalSimulationTimeD = (double)(totalSimulationTime);
float CPU_util = (float)(busyTime/totalSimulationTimeD);
System.out.printf(format:"CPU Utilization: %.3f", CPU_util);
```

To calculate the CPU utilization, I divide these 2 values and store the utilization in **CPU_util.**

- **Throughput:** This is the rate at which patrons are served.

```
private long completionTime;
```

To calculate this, I created a variable in the patron class which stores the time for when they have finished their last drink **completionTime.**

```
//This method calculates the throughput over the entire simulation on a specific window time frame
public static List<Double> calculateThroughputOverTime(List<Long> completionTimes) {


    List<Double> throughputs = new ArrayList<>();

    //get the start time of the first and last completion time (so that we can loop through the times)
    long startTime = completionTimes.get(index:0);
    long endTime = completionTimes.get(completionTimes.size() - 1);

    //loop from start time to end time and calculate throughput for every windowSize (5s in this case)
    for (long windowStart = startTime; windowStart < endTime; windowStart += 1000) {
        long windowEnd = windowStart + windowSize;
        int count = 0;

        //if the completion time falls within the time frame add 1 to count
        for (Long time : completionTimes) {
            if (time >= windowStart && time < windowEnd) {
                count++;
            } else if (time >= windowEnd) {
                break; //we are out of the time frame we are checking so leave
            }
        }

        double throughput = count / (windowSize / 1000.0); // orders per second
        throughputs.add(throughput);
    }

    return throughputs;
}
```

Then I created a method in the SchedulingSimulation class that loops through the first completion time to the last completion time. In that loop, I create windows the size of 1000ms

(1s), on which I check to see if there are any other patrons that finished their drink in that time period. If so, I increment count. I repeat this for all windows and then have a throughput value for each window (which is the size of 3s) stored in the list **throughputs**.

Finding values for s (context switch) and q (time quantum):

In this section of my report, I'm going to explain how I got to my values for s and q (1 and 100 respectively).

Finding s: 1

To find a suitable value for s, I ran a shell script (findS.sh) that runs the program multiple times (keeping the other parameters constant), finds and outputs all the averages of the metrics to a .csv file (best_s_results.csv). I inspected these values and chose the best one to use:

best_s_results.csv > data

```csv
Context_Switch(s),Avg_Response_Time,Avg_Waiting_Time,Avg_Turnaround_Time,CPU_Utilization,Avg_Throughput
1,2117.86,12716.22,13194.82,0.963442196858402,2.9411764705882355
2,2347.3,14677.04,15160.58,0.928991980509593,2.6315789473684212
3,1863.38,13066.62,13546.08,0.9102730117621668,2.7777777777777777
4,2303.82,13592.58,14077.66,0.8927153038463631,2.7777777777777777
5,2016.24,14536.58,15012.58,0.8742001091323974,2.5
10,2733.26,18696.6,19208.9,0.7350285459772442,2.0833333333333335
20,3118.52,23678.9,24169.9,0.5890676296782665,1.6666666666666667
30,4520.58,26017.58,26500.04,0.5877174659067173,1.5151515151515151
50,4694.76,34040.52,34535.66,0.42796341573847685,1.1363636363636365
```

From these values, it is clear to me that 1 is the best value to use for s. As we increase s, the **CPU utilization** and **throughput** decreases. The **response time**, **waiting time** and **turnaround time** increases as we increase s. We are trying to maximize the **CPU utilization** and **throughput** and minimize the **response time**, **waiting time** and **turnaround time.** For this reason, I chose 1 as my suitable context switching time.

Finding q: 100

To find a suitable value for q, I ran a shell script (findQ.sh) that is quite similar to findS.sh in that it runs the program multiple times while keeping all other parameter arguments constant. It also finds and outputs all the averages of the metrics to a .csv file (best_q_results.csv) similar to that of the previous example. I inspected these values and chose the best one to use:

best_q_results.csv > data

```csv
Quantum(q),Avg_Response_Time,Avg_Waiting_Time,Avg_Turnaround_Time,CPU_Utilization,Avg_Throughput
1,6257.62,27618.04,28069.64,0.4973172,1.3888888888888888
5,2686.62,12535.64,12977.02,0.803939,2.7777777777777777
10,2745.34,17351.74,17858.96,0.9012518,2.1739130434782608
25,2115.74,11349.1,11806.7,0.9346532,3.125
50,1688.08,12011.22,12508.52,0.9489555,3.125
69,1693.56,10528.82,11024.62,0.959495,3.3333333333333335
75,1851.92,12241.06,12702.44,0.95215565,2.9411764705882355
85,1842.1,11820.28,12277.92,0.9569039,3.125
100,1551.76,11493.4,11960.44,0.968653,3.125
125,1902.6,11496.98,11938.26,0.95410866,3.125
150,1317.2,11856.7,12324.94,0.9572597,3.125
175,1606.14,12060.02,12531.28,0.9623496,3.125
200,1473.14,11444.66,11926.6,0.9618384,3.3333333333333335
500,1482.42,11778.98,12252.48,0.9654297,3.125
```

From these values it is clear that we would want to use a q value higher than 25, to reduce the **response time**. Upon further inspection, anything above 100 makes the metrics plateau. has the greatest **CPU utilization** and **throughput.** Because 100 has the lowest **response time** and highest **CPU utilization**, I chose 100 as my suitable time quantum value.

Running my experiment:

In this section of my report, I'm going to explain how I ran the experiment to collect the data required for my graphs.

Outputting my data:

In my code, I output all my data into .csv file in a specific format, so that I can extract the necessary information easily. The data in the .csv file is in the format:

- List of the response times of the patrons (R_p) with average at the end – [R_1, R_2...R_{50},R_{avg}]
- List of the waiting times of the patrons (W_p) with average at the end – [W_1, W_2...W_{50},W_{avg}]
- List of the turnaround times of the patrons (T_p) with average at the end – [T_1, T_2...T_{50},T_{avg}]
- [Busy time, idle time, total simulation time, CPU utilization]
- List of the throughput for each window – [window 1, window 2... window n]

Collecting my data:

I ran the program multiple times with different seeds for each algorithm. (the same 10 seeds were used across all algorithms). I made sure to run the program with 50 patrons, so that I could get a wide range of values (with multiple windows for throughput). I used this data to then create a final average for each algorithm

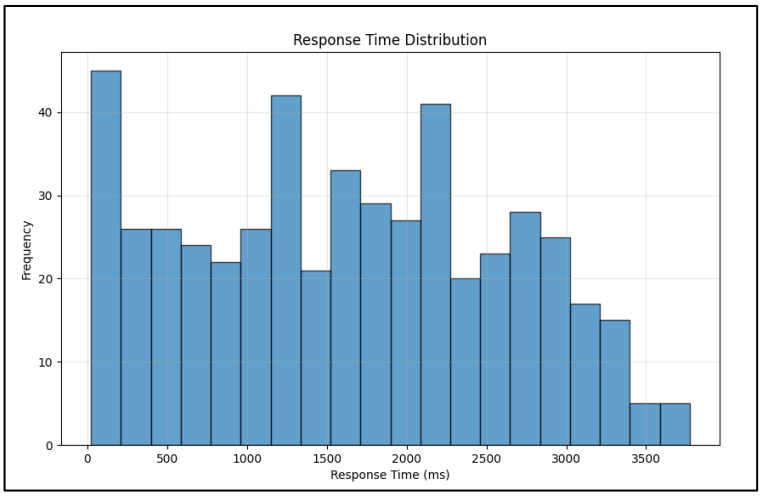
SJF_results_seed_2.csv		2025/05/04 21:47	Comma Separate...	1 KB
SJF_results_seed_7.csv		2025/05/04 21:47	Comma Separate...	1 KB
SJF_results_seed_9.csv		2025/05/04 21:47	Comma Separate...	1 KB
SJF_results_seed_10.csv		2025/05/04 21:49	Comma Separate...	1 KB
SJF_results_seed_12.csv		2025/05/04 21:48	Comma Separate...	1 KB
SJF_results_seed_34.csv		2025/05/04 21:47	Comma Separate...	1 KB
SJF_results_seed_42.csv		2025/05/04 21:46	Comma Separate...	1 KB
SJF_results_seed_45.csv		2025/05/04 21:48	Comma Separate...	1 KB
SJF_results_seed_57.csv		2025/05/04 21:46	Comma Separate...	1 KB
SJF_results_seed_99.csv		2025/05/04 21:48	Comma Separate...	1 KB
SJF_resultsavg.csv		2025/05/04 21:54	Comma Separate...	2 KB

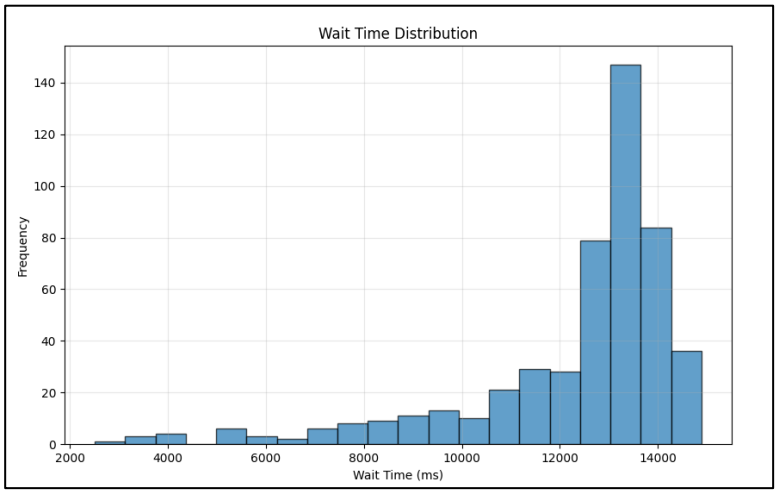
Processing my data:

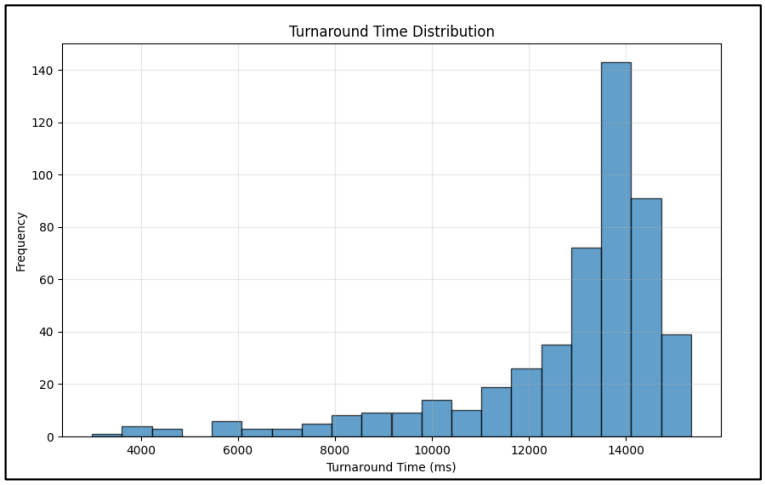
In this section of my report, I'm going to discuss the graphs of the data I collected. I will use this data to find which algorithm best suits the roll of a barman.

Distribution:

FCFS:

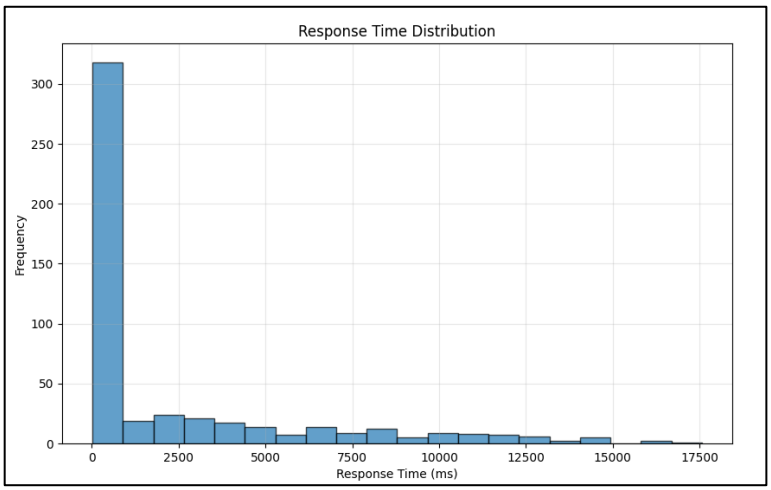


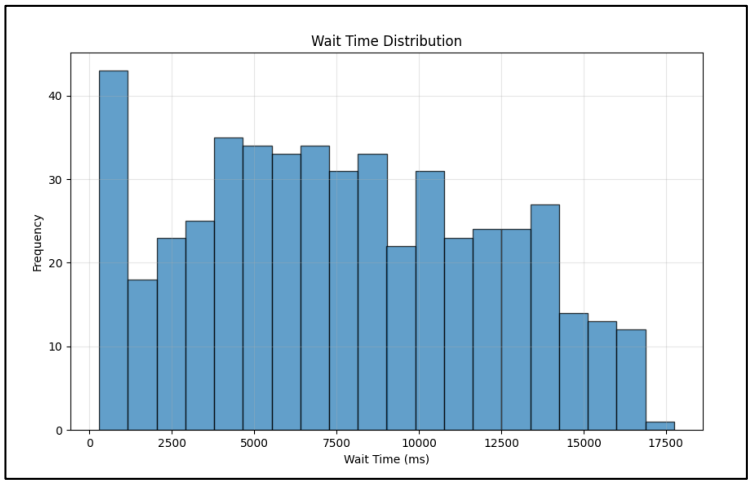


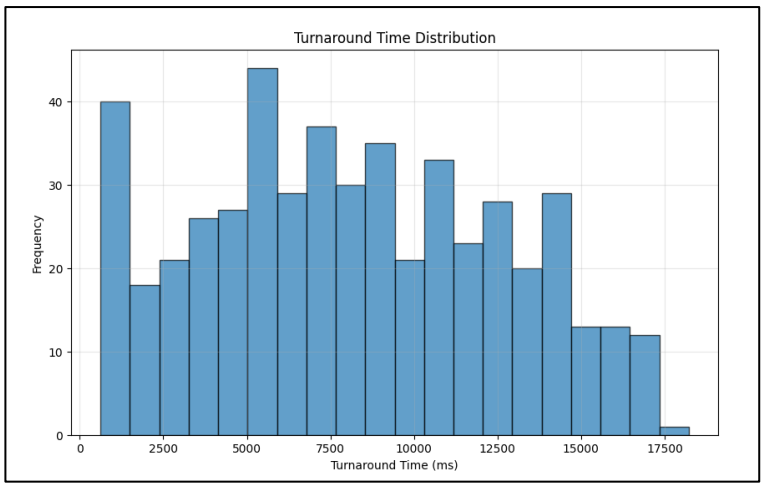


- The response time across the board is similar. The median value is between 0-200ms.
- The wait time is skewed to the right. The median value is between 12500-13500 ms. This makes sense as FCFS only serves who comes first, so other jobs must wait.
- The turnaround time is also skewed to the right. The median value is between 13500 – 14500 ms. Same reasoning as before for this.

SJF:

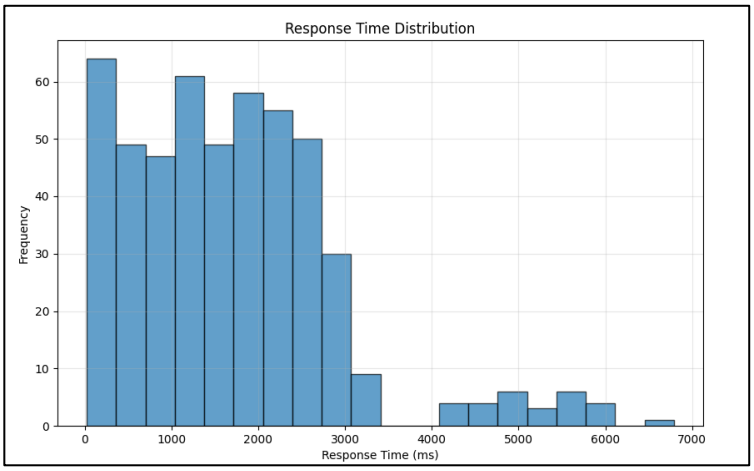


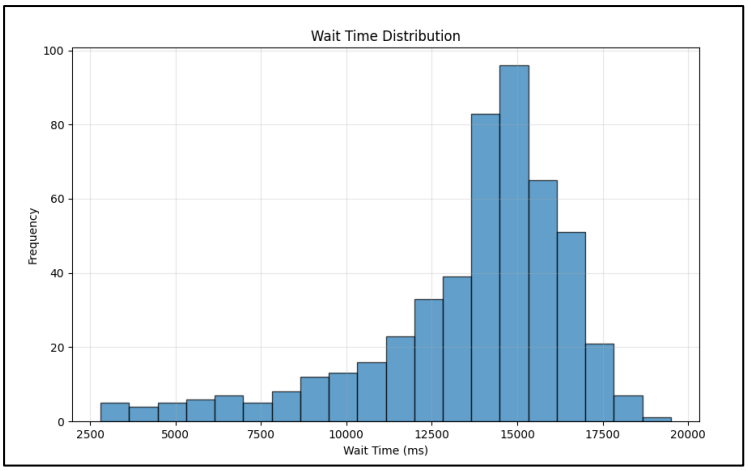


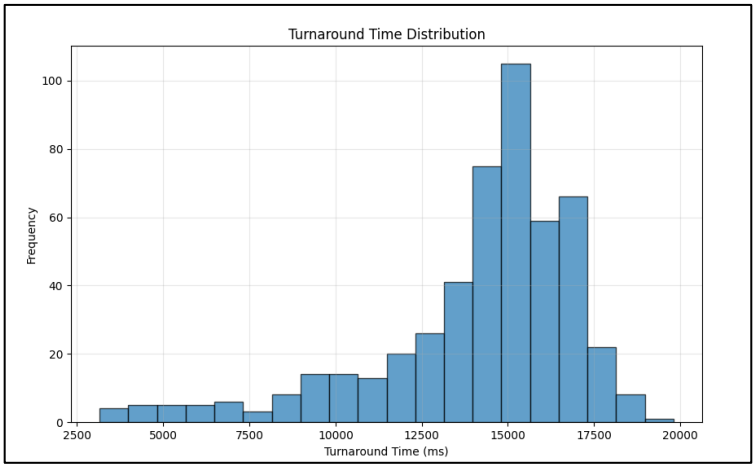


- The response time is heavily skewed to the left. The median value is 0-850ms. This makes sense if a lot of the jobs given were short, as SJF would help them immediately.
- The waiting time and turnaround time distributions are quite similar.
- Waiting time median: 0 – 850 ms.
- Turnaround time median: 5000-5850 ms.

RR:







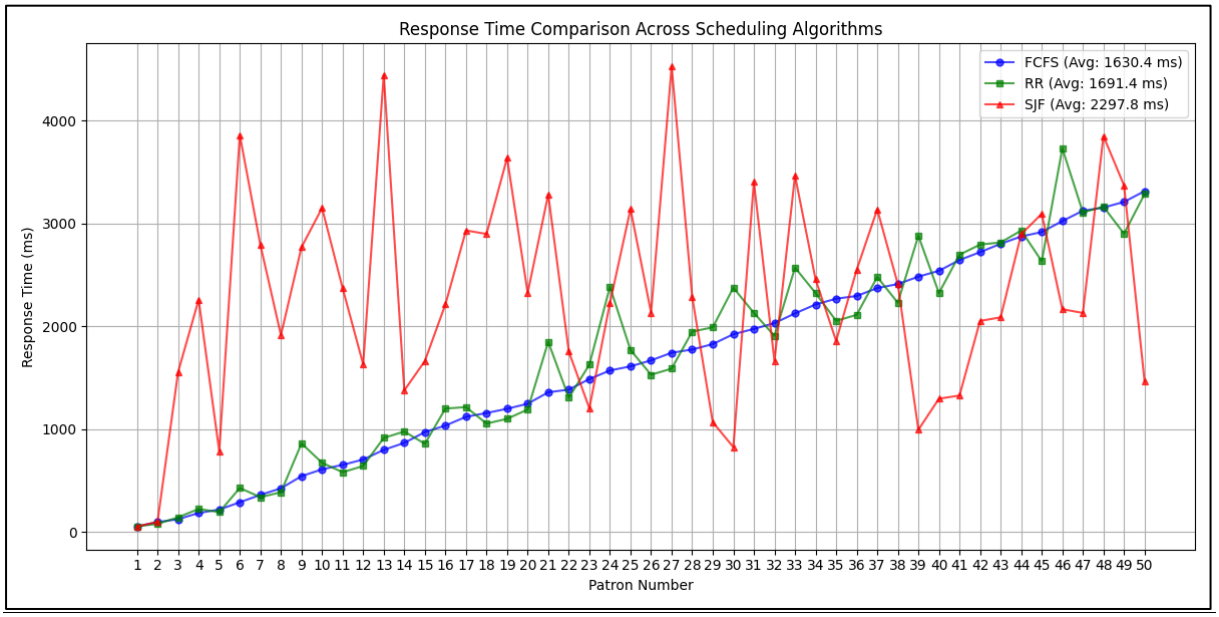
- The response time is skewed heavily to the left, with almost nothing on the right. This is because if the barman is preparing a drink and a new patron arrives while an interrupt takes place, it will serve the new patron. The median is between 0-250 ms.
- The waiting time and turnaround time distribution follows a normal distribution with a median around 15000 ms.

Distribution conclusion:

The **response time median** for FCFS and RR is low, which makes it the best. As a customer at a bar, a quick response time is a good thing. **The turnaround and waiting time** for SJF is better than FCFS and RR.

Patrons vs metrics:

Response time:

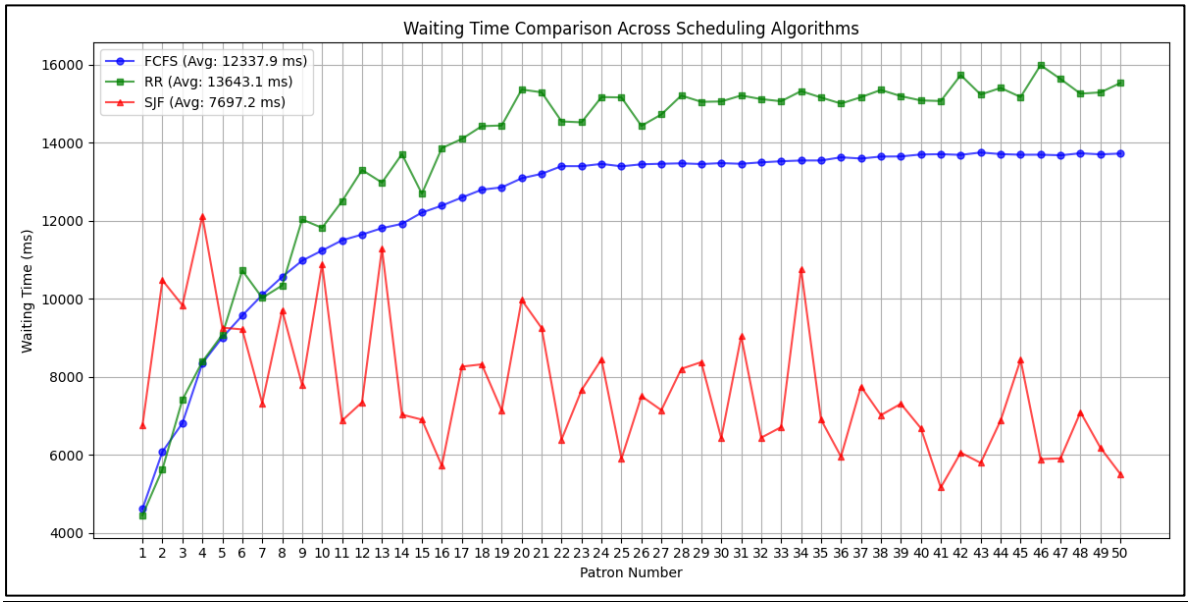


(combined_response_times.png) – in zip

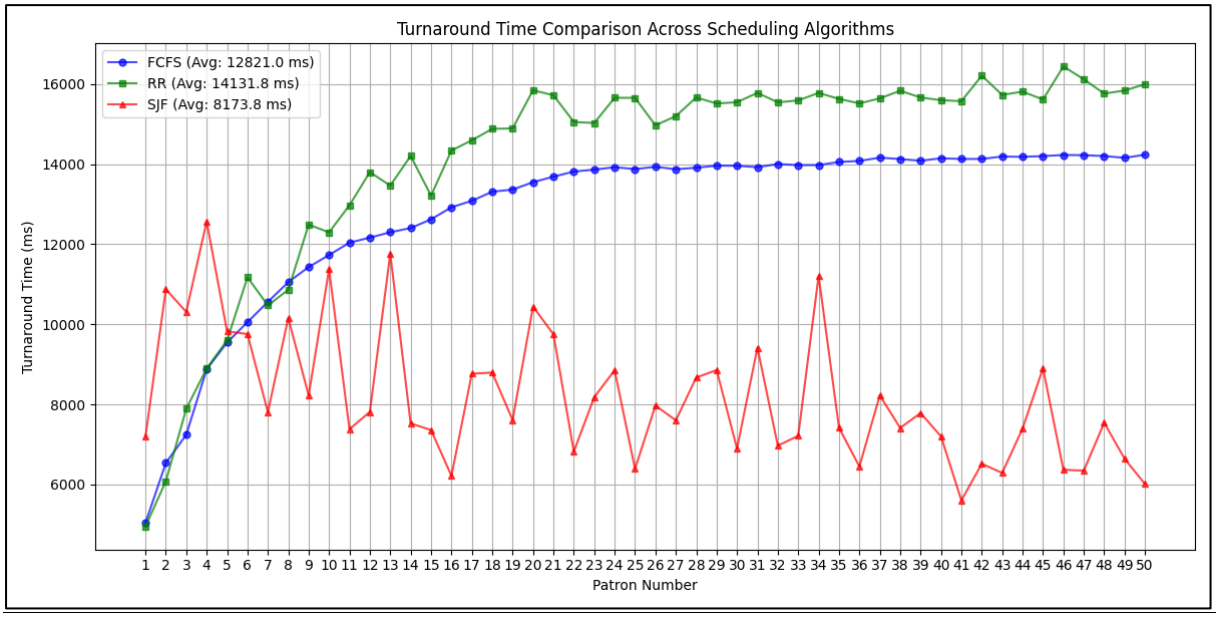
- **FCFS:** The response time gradually increases as more patrons enter the bar.
- **RR:** Similar to FCFS the response time gradually increases as more patrons enter the bar, with some dipping under or over the expected line. (Probably due to interrupts)
- **SJF:** Response time is higher than RR and FCFS, being low when a patron enters and orders a drink that won't take long to prepare.

Waiting and turnaround time:

Because the graphs for both waiting and turnaround time follow the same patterns, I will be discussing their results together. As the patrons increase and the graphs plateau there is a possibility that the threads of FCFS and RR could be experiencing starvation.



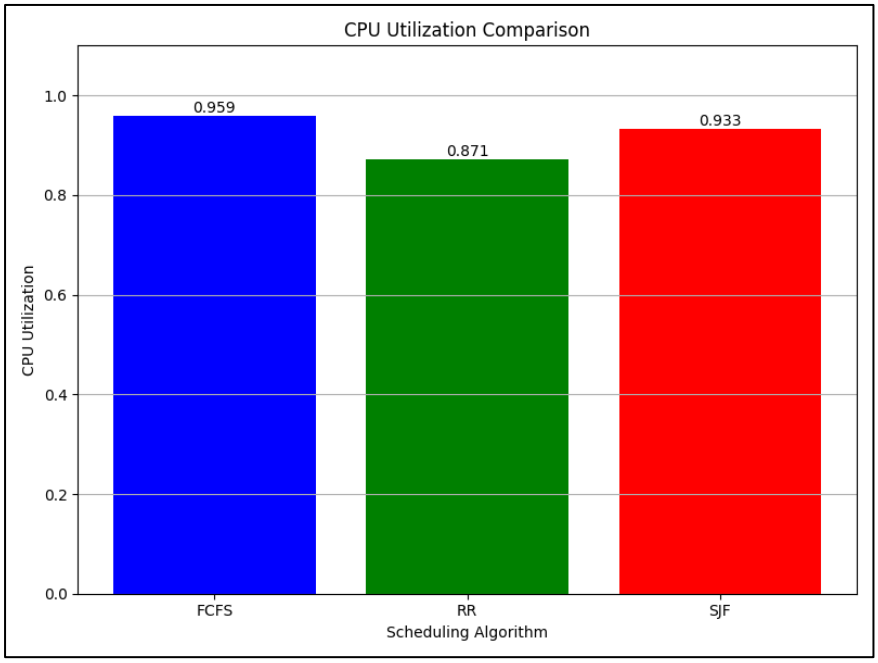
(combined_waiting_times.png) – in zip



(combined_turnaround_times.png) – in zip

- **FCFS:** The waiting and turnaround time gradually increases as more patrons enter the bar and plateaus at 20.
- **RR:** Similar to FCFS the waiting and turnaround time gradually increases as more patrons enter the bar, with some dipping under or over the expected line. The line also plateaus at 20. The response time is a bit slower than FCFS.
- **SJF:** Waiting and turnaround time follows an average time with a lot of variance. The mean seems to be well below FCFS and RR (but only for a large number of patrons).

CPU Utilization:

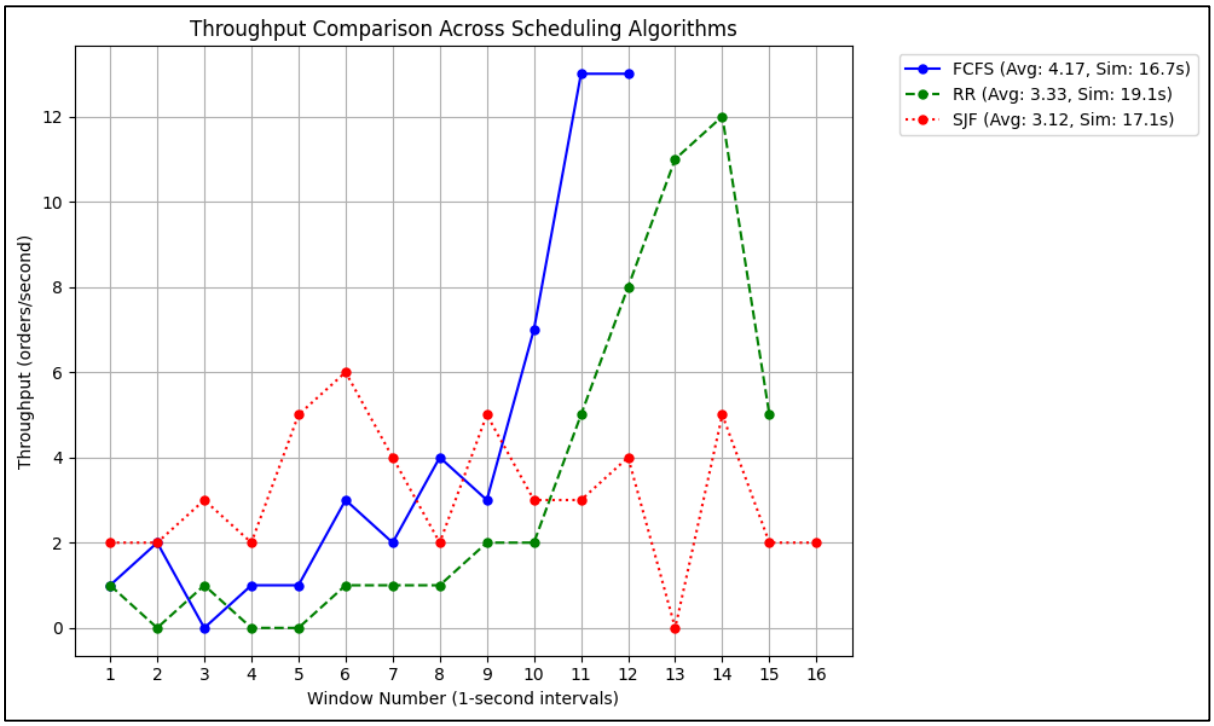


(cpu_utilization.png) – in zip

- **FCFS:** This has the highest CPU utilization with 95.9%
- **RR:** This has the lowest highest CPU utilization with 87.1%
- **SJF:** This has the second highest CPU utilization with 93.3%
- From this I can take that the FCFS barman works the hardest.

Throughput:

I calculated the throughput using a sliding window that would be 1s (1000ms) long. I will compare the values based on the averages and what they looked like at the 8th window (the middle of the barman's shift). (Avg – average, Sim – total simulation time)



(cpu_throughput.png) – in zip

- **FCFS:** On average, the throughput is the highest at 4.17 patrons/second. At the 8th window it is also the highest. The throughput of FCFS seems to spike towards the end, and as you can see, its simulation time is the shortest at 16.7s.
- **RR:** Follows a similar pattern to FCFS. The average through put is 3.3 patrons/second. At window 8 RR is the lowest in terms of throughput. RR the worst with a total simulation time of 19.1s.
- **SJF:** Throughput follows a straight line staying steady throughout the simulation with an average of 3.12 patrons/second. At the 8th window SJF is slightly better than RR, which holds true with my data because SJF has the 2nd best total simulation time with 17.1s.

Conclusion:

In conclusion, the algorithms behave as expected. FCFS and RR are quite similar in the values they produced, whereas SJF is a bit sporadic with a large variance from the mean value.

First-Come, First-Serve Scheduling will work best for the purpose of being a barman. The FCFS bartender is the hardest working (**biggest CPU utilization**) and has the highest work rate (**best throughput**) out of all the employees we tested. Customers have also said that whenever FCFS is working, they always receive attention when they arrive early (**has the best response time**). For these reasons, I would recommend FCFS.